

1) LCM GCD

```
package main
import "fmt"
func lcm(temp1 int,temp2 int) {
var lcmnum int =1
if(temp1>temp2) {
lcmnum=temp1 }else{
lcmnum=temp2}
for {
if(lcmnum%temp1==0 && lcmnum%temp2==0) { // And operator
fmt.Printf("LCM of %d and %d is %d",temp1,temp2,lcmnum)
break}
lcmnum++} return }
func gcd(temp1 int,temp2 int){
var gcdnum int
for i := 1; i <=temp1 && i <=temp2 ; i++ {
if(temp1%i==0 && temp2%i==0) {
gcdnum=i}}
fmt.Printf("GCD of %d and %d is %d",temp1,temp2,gcdnum)
return}
func main() {
var n1,n2,action int
fmt.Println("Enter two positive integers : ")
fmt.Scanln(&n1)    fmt.Scanln(&n2)
fmt.Println("Enter 1 for LCM and 2 for GCD")
fmt.Scanln(&action)
switch action {
case 1:
lcm(n1,n2)
case 2:
gcd(n1,n2)}}
-----
```

-

*** 11array revers sort***

```
package main
import (
    "fmt"
    "sort")
func main() {
    fmt.Println("Interger Reverse Sort")
    num := []int{50, 90, 30, 10, 50}
    sort.Sort(sort.Reverse(sort.IntSlice(num)))
    fmt.Println(num)
    fmt.Println()
    fmt.Println("String Reverse Sort")
    text := []string{"Japan", "UK", "Germany", "Australia", "Pakistan"}
    sort.Sort(sort.Reverse(sort.StringSlice(text)))
    fmt.Println(text)
}
-----
```

--

**** 12comprising of contains****

```
ackage main
import (
    "fmt"
    "strings")
func main() {
```

```

    fmt.Println(strings.ContainsAny("Germany", "G"))
    fmt.Println(strings.ContainsAny("Germany", "g"))
    fmt.Println(strings.Contains("Germany", "Ger"))
    fmt.Println(strings.Contains("Germany", "ger"))
    fmt.Println(strings.Contains("Germany", "er"))
    fmt.Println(strings.Count("cheese", "e"))
    fmt.Println(strings.EqualFold("Cat", "cAT"))
    fmt.Println(strings.EqualFold("India", "Indiana"))
}

```

****wk6 addition of 2 num****

```

package main
import "fmt"
func main() {
    fmt.Print("Enter First String: ")
    var first string
    fmt.Scanln(&first)
    fmt.Print("Enter Second String: ")
    var second string
    fmt.Scanln(&second)
    fmt.Print(first + second)
}

```

*****wk9 caculate avg using array****

```

package main
import "fmt"
func main() {
    var num [100]int
    var temp, sum, avg int
    fmt.Print("Enter number of elements: ")
    fmt.Scanln(&temp)
    for i := 0; i < temp; i++ {
        fmt.Print("Enter the number : ")
        fmt.Scanln(&num[i])
        sum += num[i]
    }
    avg = sum / temp
    fmt.Printf("The Average of entered %d number(s) is%d", temp, avg)
}

```

**** wk4 calculate sd math package*****

```

package main
import (
    "fmt"
    "math"
)
func main(){
    var num[10] float64
    var sum,mean,sd float64
    fmt.Println("***** Enter 10 elements *****")
    for i := 1; i <= 10; i++ {
        fmt.Printf("Enter %d element : ",i)
        fmt.Scan(&num[i-1])
        sum += num[i-1]
    }
    mean = sum/10;
    for j := 0; j < 10; j++ {
        // The use of Pow math function func Pow(x, y float64)
        float64
    }
}

```

```

}
sd += math.Pow(num[j] - mean, 2)
// The use of Sqrt math function func Sqrt(x float64) float64
sd = math.Sqrt(sd/10)
fmt.Println("The Standard Deviation is : ",sd)
}
-----

*****wk5 floyds triangle*****
package main
import "fmt"
func main(){
var rows int
var temp int = 1
fmt.Print("Enter number of rows : ")
fmt.Scan(&rows)
for i := 1; i <= rows; i++ {
for k := 1; k <= i; k++ {
fmt.Printf(" %d",temp)
temp++
}
fmt.Println("")
}}
-----

**** wk10 delete duplicate*****
package main
import "fmt"
func removeDuplicates(s []string) []string
bucket := make(map[string]bool)
var result []string
for _, str := range s {
if _, ok := bucket[str]; !ok {
bucket[str] = true
result = append(result, str)
}
}
return
func main() {
// creating an array of strings
:= []string{"abc", "cde", "efg", "efg", "abc", "cde"}
fmt.Println("The given array of string is:", array)
fmt.Println()
// calling the function
:= removeDuplicates(array)
fmt.Println("The array obtained after removing the duplicate entries
is:", result)
}
-----

*****wk 3 from another package*****
package main
import (
parent "family/father"
child "family/father/son"
"fmt")
func main() {
f := new(parent.Father)
fmt.Println(f.Data("Mr. Jeremy Maclin"))
c := new(child.Son)
fmt.Println(c.Data("Riley Maclin"))}

```

```

package father
import "fmt"
func init() {
fmt.Println("Father package initialized")}
type Father struct {
Name string}
func (f Father) Data(name string) string {
f.Name = "Father : " + name
return f.Name}
package son
import "fmt"
func init() {
fmt.Println("Son package initialized")}
type Son struct {
Name string}
func (s Son) Data(name string) string {
s.Name = "Son : " + name
return s.Name
}
-----
**** wk7 string is Palindrome or not****
package main
import "fmt"
func main() {
fmt.Println("Golang program to check for palindrome")
var number,rem,temporary int
var reverse int = 0
number = 45454
temporary=number
for{
rem = number%10
reverse = reverse*10 + rem
number /= 10
if(number==0){
break
}
}
if(temporary==reverse){
fmt.Printf("Number %d is a Palindrome",temporary)
}else{
fmt.Printf("Number %d is not a Palindrome",temporary)
}
}
-----
**** wk 14 CRUD using MYSQL****
package main
import (
"fmt"
"runtime"
"sync"
)
func main() {
runtime.GOMAXPROCS(3)
var processTest sync.WaitGroup
processTest.Add(3)
go func() {
defer processTest.Done()
for i := 0; i < 30; i++ {
for j := 51; j <= 100; j++ {

```

```

fmt.Printf(" %d", j)
if j == 100{
fmt.Println()
}}}}()
go func() {
defer processTest.Done()
for j := 0; j < 10; j++ {
for char := 'A'; char < 'A'+26; char++ {
fmt.Printf("%c ", char)
if char == 'Z' {
fmt.Println()
}}}}()
go func() {
defer processTest.Done()
for i := 0; i < 30; i++ {
for j := 0; j <= 50; j++ {
fmt.Printf(" %d", j)
if j == 50 {
fmt.Println()
}}}}()
processTest.Wait()
}
-----
****Wk 2 pyramid of num****
package main
import "fmt"
func main(){
var n int
fmt.print("enter the number of levels")
fmt.scan(&n)
for i:=1;i<=n;i++){
for j:=1;j<=n-i;j++){
fmt.print("")}
for j:=1;j<=i;j++){
fmt.print(j)}
fmt.println()
}}
*****

```